

---

## PORTFOLIO: PART 2

---

# UR3e Motion planning in ROS2 for battery swapping system

AIS4104 - Robotics and Intelligent Systems with Project

Master in Mechatronics and Automation  
Spring 2025

Faculty of Information Technology and Electrical Engineering (IE)  
Norwegian University of Science and Technology (NTNU)

Jon Urcelay San Roman

29th May 2025

## Abstract

This project focused on the development of a drone battery swapping system based on an UR3e robotic arm. For this, ROS will be used to develop a real-time control system capable of performing the task without human intervention. The main objective of the project includes defining and setting up a simplified physical workspace, as well as all the necessary hardware and software tools.

# Contents

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>3</b> |
| <b>2</b> | <b>Theoretical background</b>                       | <b>4</b> |
| 2.1      | Transformation Matrices . . . . .                   | 4        |
| 2.2      | Quaternions from Roll-Pitch-Yaw . . . . .           | 4        |
| 2.3      | Forward Kinematics of a 6-DOF Robotic Arm . . . . . | 4        |
| 2.4      | Inverse Kinematics of a 6-DOF Robotic Arm . . . . . | 5        |
| 2.5      | Motion Planning . . . . .                           | 6        |
| 2.6      | Trajectory Planning . . . . .                       | 6        |
| <b>3</b> | <b>Materials and methods</b>                        | <b>8</b> |
| 3.1      | Hardware . . . . .                                  | 8        |
| 3.1.1    | UR3e Robotic Arm . . . . .                          | 8        |
| 3.1.2    | OnRobot 2FG7 Gripper . . . . .                      | 8        |
| 3.1.3    | L515 Realsense Camera . . . . .                     | 9        |
| 3.1.4    | DFROBOT FIT0729 Camera . . . . .                    | 10       |
| 3.2      | Software . . . . .                                  | 10       |
| 3.2.1    | ROS 2 Jazzy . . . . .                               | 10       |
| 3.2.2    | Windows Subsystem for Linux . . . . .               | 11       |
| 3.2.3    | VirtualBox and VMware . . . . .                     | 11       |
| 3.2.4    | Ubuntu Noble 24.04 . . . . .                        | 12       |
| 3.2.5    | RViz 2 . . . . .                                    | 12       |
| 3.2.6    | Moveit 2 . . . . .                                  | 12       |
| 3.2.7    | ArUco . . . . .                                     | 13       |
| 3.2.8    | YOLOv8 . . . . .                                    | 14       |
| 3.2.9    | Siemens NX and PrusaSlicer . . . . .                | 14       |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>4</b> | <b>Results</b>                               | <b>15</b> |
| 4.1      | Simplified station setup . . . . .           | 15        |
| 4.1.1    | 3D printed components . . . . .              | 16        |
| 4.1.2    | Devices and communication . . . . .          | 17        |
| 4.1.3    | Software setup on PC . . . . .               | 18        |
| 4.1.4    | Software setup on PolyScope . . . . .        | 19        |
| 4.2      | ROS2 code . . . . .                          | 19        |
| 4.2.1    | Overview . . . . .                           | 19        |
| 4.2.2    | Interfaces . . . . .                         | 21        |
| 4.2.3    | Package structure and dependencies . . . . . | 23        |
| 4.2.4    | Observations . . . . .                       | 24        |
| <b>5</b> | <b>Discussion</b>                            | <b>25</b> |
| <b>6</b> | <b>Conclusion</b>                            | <b>26</b> |
| <b>7</b> | <b>Future Work</b>                           | <b>27</b> |
|          | <b>References</b>                            | <b>28</b> |

## List of Figures

|    |                                                                                                                         |    |
|----|-------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Illustration of the DH parameters. . . . .                                                                              | 5  |
| 2  | UR3e robotic arm, control box and teach pendant. . . . .                                                                | 8  |
| 3  | 2FG7 Gripper, quick changer and UR3e with the gripper assembled. . . . .                                                | 9  |
| 4  | Intel Realsense L515 LiDAR Camera. . . . .                                                                              | 9  |
| 5  | DFROBOT FIT0729 USB 2.0 Camera. . . . .                                                                                 | 10 |
| 6  | Moveit 2 running on RViz 2. . . . .                                                                                     | 13 |
| 7  | ArUco markers with pose estimation. . . . .                                                                             | 13 |
| 8  | Siemens NX (left) and PrusaSlicer (right). . . . .                                                                      | 14 |
| 9  | 3D representation of the simplified station. . . . .                                                                    | 15 |
| 10 | Simplified station physical setup. . . . .                                                                              | 16 |
| 11 | 3D printed components. . . . .                                                                                          | 17 |
| 12 | 3D printed batteries clip connection. . . . .                                                                           | 17 |
| 13 | USB hub connection: PC, camera & UR3e (with gripper). . . . .                                                           | 18 |
| 14 | Visual representation of YOLOv8 model's prediction. . . . .                                                             | 22 |
| 15 | Visual representation of ArUco's pose detection and established axes (red: x,<br>green: y, blue: z). . . . .            | 23 |
| 16 | Visual representation of hardcoded relative poses A, B and C for YOLOv8 value<br>2 (red: x - green y - blue z). . . . . | 23 |

## List of Tables

|   |                                               |    |
|---|-----------------------------------------------|----|
| 1 | Custom developed packages. . . . .            | 24 |
| 2 | External packages pulled from GitHub. . . . . | 24 |
| 3 | Packages installed from binaries. . . . .     | 24 |

## Acronyms

**DOF** Degrees Of Freedom.

**RPY** Roll-Pitch-Yaw.

**FK** Forward Kinematics.

**DH** Denavit-Hartenberg.

**IK** Inverse Kinematics.

**RRT** Rapid-exploring Random Trees.

**PRM** Probabilistic Roadmaps.

**PTP** Point-to-Point.

**LIN** Linear Trajectory.

**SLERP** Spherical Linear Interpolation.

**UR** Universal Robots.

**CAD** Computer Assisted Design.

**GUI** Graphical User Interface.

**URCap** Universal Robots Capability.

**LiDAR** Light Detection and Ranging.

**RGB** Red Green Blue.

**FPS** Frames Per Second.

**ROS** Robot Operating System.

**DDS** Data Distribution Service.

**WSL** Windows Subsystem for Linux.

**VM** Virtual Machine.

**OS** Operating System.

**OMPL** Open Motion Planning Library.

**FCL** Flexible Collision Library.

**KDL** Kinematics and Dynamics Library.

**PnP** Perspective-n-Point.

**YOLO** You Only Look Once.

**IP** Internet Protocol.

**TCP** Transmission Control Protocol.

**NAT** Network Address Translation.

**LAN** Local Area Network.

**DHCP** Dynamic Host Configuration Protocol.

**URDF** Unified Robot Description Format.

**SRDF** Semantic Robot Description Format.

**STL** Stereo Lithography.

# 1 Introduction

An automatic drone battery swapping station is a mechanism that changes the battery of a drone without human intervention, removing the low-charge battery from the drone and replacing it with a fully charged one.

For this project, the objective is to develop a simplified prototype of the station, using a ROS controlled 6 Degrees of Freedom (DOF) UR3e robotic arm. For this, the project focuses on the following two tasks:

1. Set up a simplified physical station, including the necessary hardware and connections.
2. Develop the ROS architecture that will control the robot motion planning and integrate it with computer vision.

## 2 Theoretical background

### 2.1 Transformation Matrices

Transformation matrices describe the position and orientation of a rigid body in 3D space, essential for robotic arm control. A homogeneous transformation matrix  $T \in SE(3)$  combines a rotation matrix  $R \in SO(3)$  and a translation vector  $p \in \mathbb{R}^3$ :

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \quad R^T R = I, \quad \det(R) = 1.$$

Given two frames  $\{A\}$  and  $\{B\}$ ,  $T_A^B$  maps points from frame  $\{A\}$  to  $\{B\}$ . Composition of transformations follows  $T_A^C = T_A^B T_B^C$  [1].

### 2.2 Quaternions from Roll-Pitch-Yaw

Quaternions represent rotations to avoid singularities in Euler angles, such as roll-pitch-yaw (RPY). A unit quaternion  $q = [w, x, y, z]$ , with  $w^2 + x^2 + y^2 + z^2 = 1$ , encodes a rotation. For RPY angles  $(\phi, \theta, \psi)$ , corresponding to Euler rotations about  $z$ ,  $y$  and  $x$ , the rotation matrix is:

$$R = R_z(\phi)R_y(\theta)R_x(\psi) = \begin{bmatrix} R_{11} & R_{12} & R_{13} \\ R_{21} & R_{22} & R_{23} \\ R_{31} & R_{32} & R_{33} \end{bmatrix} = \begin{bmatrix} c_\phi c_\theta & c_\phi s_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta c_\psi + s_\phi s_\psi \\ s_\phi c_\theta & s_\phi s_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta c_\psi - c_\phi s_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix},$$

where  $c_\theta = \cos \theta$ ,  $s_\theta = \sin \theta$ . The quaternion is derived as:

$$w = \sqrt{1 + R_{11} + R_{22} + R_{33}}/2, \quad x = \frac{R_{32} - R_{23}}{4w}, \quad y = \frac{R_{13} - R_{31}}{4w}, \quad z = \frac{R_{21} - R_{12}}{4w}.$$

Quaternions are used in ROS2 for smooth orientation interpolation [1].

### 2.3 Forward Kinematics of a 6-DOF Robotic Arm

Forward kinematics (FK) maps joint angles  $\theta = [\theta_1, \dots, \theta_6] \in \mathbb{R}^6$  to the end-effector pose  $T \in SE(3)$  for a 6-DOF robotic arm. Using the Denavit-Hartenberg (DH) convention, each joint  $i$  has parameters  $(a_i, \alpha_i, d_i, \theta_i)$ , where  $a_i$  is the link length (distance from joint axis  $i - 1$  to  $i$  along the common perpendicular),  $\alpha_i$  is the link twist (angle between joint axes  $i - 1$  and  $i$ ),  $d_i$  is the link offset (distance along axis  $i - 1$  to the common perpendicular), and  $\theta_i$  is the joint angle (rotation about axis  $i - 1$ ). See Figure 1 for a visual representation.

These parameters define the transformation:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The end-effector pose is  $T_0^6 = T_0^1 T_1^2 \dots T_5^6$ . DH parameters specify link lengths and joint offsets, computed numerically in ROS2 [1].

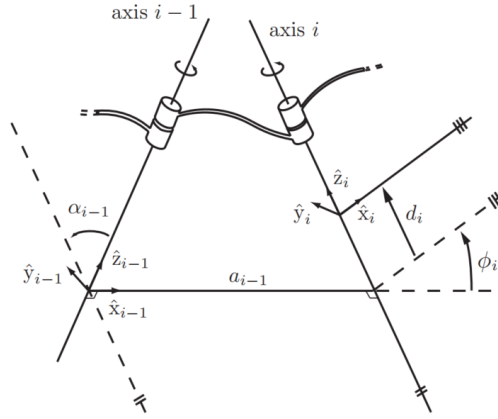


Figure 1: Illustration of the DH parameters.

## 2.4 Inverse Kinematics of a 6-DOF Robotic Arm

Inverse kinematics (IK) computes joint angles  $\theta = [\theta_1, \dots, \theta_6] \in \mathbb{R}^6$  given a desired end-effector pose  $T_d \in SE(3)$ . For a 6-DOF arm, up to eight solutions may exist due to joint limits, workspace boundaries and singularities.

The IK problem is solved iteratively using the Jacobian matrix  $J(\theta) \in \mathbb{R}^{6 \times 6}$ , which relates joint velocities  $\dot{\theta}$  to the end-effector twist  $V = [v, \omega]^T \in \mathbb{R}^6$ , where  $v$  is the linear velocity and  $\omega$  is the angular velocity:

$$V = J(\theta)\dot{\theta}, \quad J(\theta) = \begin{bmatrix} J_v(\theta) \\ J_\omega(\theta) \end{bmatrix}.$$

The Jacobian's linear part  $J_v(\theta)$  maps joint velocities to end-effector translation, while the angular part  $J_\omega(\theta)$  maps to rotation, computed from joint axes and link vectors. Near singularities, where  $\det(J(\theta)) \approx 0$ , the Jacobian becomes ill-conditioned, requiring damping techniques like Levenberg-Marquardt [1].

Numerical methods, such as Newton-Raphson or gradient descent [1], minimize the pose error  $\|T(\theta) - T_d\|$ , often using the pseudo-inverse  $J^+(\theta)$  for non-square Jacobians or redundant arms. More detail can be found in [1].

## 2.5 Motion Planning

Motion planning computes a collision-free, optimal path from an initial to a goal configuration in the configuration space  $\mathcal{C}$ . For a 6-DOF robotic arm,  $\mathcal{C} = \mathbb{T}^6$ , where  $\mathbb{T}$  is the joint angle range for each revolute joint. Sampling-based algorithms, such as Rapidly-exploring Random Trees (RRT), construct a tree of feasible configurations by iteratively sampling points in  $\mathcal{C}$ :

$$\text{Extend}(q_{\text{near}}, q_{\text{rand}}) = q_{\text{new}}, \quad q_{\text{new}} = q_{\text{near}} + \alpha(q_{\text{rand}} - q_{\text{near}}),$$

where  $q_{\text{near}}$  is the nearest node,  $q_{\text{rand}}$  is a random configuration, and  $\alpha$  is a step size.

Collision avoidance is achieved by defining obstacle regions  $\mathcal{C}_{\text{obs}} \subset \mathcal{C}$ , ensuring sampled configurations lie in the free space  $\mathcal{C}_{\text{free}} = \mathcal{C} \setminus \mathcal{C}_{\text{obs}}$ .

Finally, path optimization minimizes a cost function, such as:

$$\text{Cost}(q) = \text{dist}(q, \mathcal{C}_{\text{obs}}) + \lambda \text{length}(q),$$

balancing safety margins and path length.

Alternatively, Probabilistic Roadmaps (PRM) discretize  $\mathcal{C}_{\text{free}}$  into a graph of collision-free nodes connected by edges, using Dijkstra's algorithm to find the shortest path:

$$\text{Cost}(p) = \sum_{e \in p} w(e), \quad w(e) = \|q_i - q_j\|.$$

The planner terminates when a kinematically feasible path connects the initial and goal configurations [1].

## 2.6 Trajectory Planning

Trajectory planning generates a time-parameterized path  $q(t)$  in the configuration space  $\mathcal{C}$  or Cartesian space, satisfying kinematic and dynamic constraints for a 6-DOF robotic arm. A cubic spline for a joint angle  $\theta(t)$  between waypoints  $\theta_0$  at time  $t_0$  and  $\theta_1$  at  $t_1$  ensures smooth motion:

$$\theta(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3,$$

where coefficients  $a_i$  are solved to enforce continuity in position, velocity, and acceleration. Joint trajectories (Point-to-Point, PTP) interpolate joint angles in  $\mathcal{C} = \mathbb{T}^6$ , minimizing joint-space distance:

$$q(t) = q_0 + (q_1 - q_0) \frac{t - t_0}{t_1 - t_0}.$$

PTP trajectories are computationally efficient but may result in curved end-effector paths in Cartesian space. Linear trajectories (LIN) interpolate the end-effector pose in Cartesian space for straight-line motion, requiring inverse kinematics at each time step:

$$p(t) = p_0 + (p_1 - p_0) \frac{t - t_0}{t_1 - t_0}, \quad R(t) = \text{slerp}(R_0, R_1, \frac{t - t_0}{t_1 - t_0}),$$

where  $p(t)$  is the position and  $R(t)$  is the orientation interpolated via spherical linear interpolation (slerp). LIN ensures precise end-effector paths but increases computational cost due to repeated inverse kinematics [1].

## 3 Materials and methods

This chapter describes the tools or methods that have been used during the development of this project. These tools are divided in two groups, hardware (Chapter 3.1) and software (Chapter 3.2).

### 3.1 Hardware

#### 3.1.1 UR3e Robotic Arm

The UR3e (Figure 2) is a small collaborative robotic arm developed by Universal Robots, being the smallest member of Universal Robots' e-Series. The 6-axis articulated arm has  $\pm 360$ -degree rotation on all wrist joints and infinite rotation on the end joint. The arm is equipped with integrated force/torque sensors and advanced safety features such as force sensing, speed monitoring and collision detection. The UR3e comes with a control box and a teach pendant (see in Figure 2), including an intuitive programming software called PolyScope GUI.

In addition, it's compatible with a wide range of peripherals, such as grippers, cameras and sensors. These peripherals are typically integrated using URCaps (Universal Robots Capability), which are software plugins for the PolyScope environment that extend the robot's capabilities with custom graphical interfaces, commands, and device drivers.

All the details about the UR3e robotic arm can be found in [2], including the datasheet or CAD files.



Figure 2: UR3e robotic arm, control box and teach pendant.

#### 3.1.2 OnRobot 2FG7 Gripper

The 2FG7 parallel gripper (Figure 3) developed by OnRobot, is a compact, versatile gripper designed for collaborative robots like the UR3e. It offers a payload capacity of 11 kg, a stroke

range of 1 mm to 73 mm and a gripping force of 20 N to 140 N. All the details for the gripper can be found in [3].

The gripper is connected to the robot using a plug-and-play coupler, called "quick changer" (see [3]), which connects the gripper to the control box using internal connections from the robotic arm.



Figure 3: 2FG7 Gripper, quick changer and UR3e with the gripper assembled.

### 3.1.3 L515 Realsense Camera

The Intel RealSense L515 LiDAR (Light Detection and Ranging) camera is a compact, high-resolution depth-sensing device designed for robotic applications (see Figure 4). Using a USB-C interface, it provides a depth resolution of up to 1024x768 at 30 frames per second (fps) and a RGB image with 1920x1080 resolution at 30 fps. All the details about the camera can be found in [4].



Figure 4: Intel Realsense L515 LiDAR Camera.

### 3.1.4 DFROBOT FIT0729 Camera

The DFROBOT FIT0729 is a compact camera module (see Figure 5). It delivers a resolution of 3280x2464 pixels at 30 fps and a 70° field of view. The camera supports adjustable focus and integrates via USB 2.0 interfaces. It has a very lightweight design and low power consumption. See all the details in [5].



Figure 5: DFROBOT FIT0729 USB 2.0 Camera.

## 3.2 Software

All the software tools used during the project are closely related to ROS, the main system that makes the robot application work. In this section a concise and basic description of all these software tools will be provided.

### 3.2.1 ROS 2 Jazzy

The Robot Operating System (ROS) is an open-source framework providing software libraries and tools to streamline robot application development. Originating in 2007, ROS has become a global standard in robotics due to its robust community and open-source model.

The initial version, ROS1, offered extensive features but faced limitations, including a lack of real-time capabilities and a centralized communication system prone to single-point failures.

To address this, ROS2 was introduced in 2015, adopting the Data Distribution Service (DDS) protocol for reliable, real-time and distributed communication. ROS2 enhances modularity, supports multi-robot systems and improves debugging, while retaining ROS1's core strengths.

This project utilizes ROS2 Jazzy distribution, selected for its status as the latest long-term support release, ensuring stability and compatibility with modern hardware, and access to updated libraries for complex robotic applications. Further details on ROS concepts are available at [6].

In ROS2, development is organized into workspaces, directories containing **src**, **build**, **install**, and **log** folders. The core workspace (underlay) stores primary ROS2 libraries, while overlay workspaces enable custom application development.

Within a custom overlay workspace, the **src** folder holds all packages, whether custom-created or pulled from Github repositories. Packages are the core units of code organization, including source code, configuration files, launch files and resources designed for sharing within the ROS2 community. Building a workspace involves compiling **src** packages using **colcon**, generating executables and dependencies in the **build** and **install** folders for deployment.

In ROS2, applications are built around **nodes**, independent modules that perform specific tasks and communicate via defined mechanisms. **Topics** enable publish-subscribe communication, where nodes publish messages to named channels and subscribers receive them asynchronously. **Services** provide request-response interactions, allowing nodes to send requests and receive replies for specific tasks. **Actions** support long-running tasks with feedback and goal-oriented communication, ideal for complex operations like robot navigation. **Parameters** allow nodes to store and retrieve configuration data dynamically. These components ensure modular, scalable, and robust communication in ROS2 applications. For further details and visual graphs, see the link referenced in [7].

### 3.2.2 Windows Subsystem for Linux

Windows Subsystem for Linux (WSL) is a compatibility layer in Windows that executes Linux binaries natively on a Windows kernel. It provides a lightweight virtualized environment, running Linux distributions with a dedicated user space and file system, without requiring a full virtual machine. WSL supports system calls, command-line interfaces, and package managers for multiple Linux distributions, integrating with Windows file systems via a 9P protocol bridge for cross-OS operations. More details can be found on [8].

### 3.2.3 VirtualBox and VMware

VirtualBox and VMware are virtualization software tools that create virtual machines (VM) to run multiple OS on a single physical computer, enabling isolated environments for developing, testing and running applications.

VirtualBox is an open-source tool, freely available, lightweight and suited for general-purpose virtualization (see more detail in [9]). In the other hand, VMware is a commercial product, providing enhanced performance, scalability and enterprise-grade features, often preferred for complex or resource-intensive applications (see more details in [10]).

### 3.2.4 Ubuntu Noble 24.04

Ubuntu Noble 24.04 is a Linux-based open-source operating system built on the Debian architecture, utilizing the Linux kernel 6.8. It manages hardware resources via kernel modules, supports application execution through a GNU user space, and handles software installation using the APT package manager with DEB packages. The system employs systemd for service management and includes a GNOME 46 desktop environment for user interaction. Find more details on its website [11].

### 3.2.5 RViz 2

RViz 2 (Figure 6) is a 3D visualization tool for ROS2, built using the Qt framework and OpenGL for rendering. It displays sensor data from ROS2 topics, such as point clouds, depth maps, and camera feeds. RViz processes messages via ROS2's rclcpp communication library, enabling real-time data visualization and debugging of robot perception and navigation. Information about RViz is provided in the ROS Jazzy documentation (see the reference in [12]).

### 3.2.6 Moveit 2

MoveIt 2 is a motion planning framework for ROS 2, implemented in C++ and Python, leveraging rclcpp and rclpy for node communication. See it in Figure 6 implemented in RViz 2. It integrates the Open Motion Planning Library (OMPL) for sampling-based planners such as Rapidly-exploring Random Trees (RRT) or Probabilistic Roadmaps (PRM).

MoveIt 2 computes forward and inverse kinematics using either the Kinematics and Dynamics Library (KDL), which uses numerical methods, or TRAC-IK, an optimization-based inverse kinematics solver designed for faster convergence and improved reliability in redundant manipulators. Joint trajectories are generated via time-parameterized splines and motion execution is handled through ROS 2 action servers. The framework supports a plugin-based architecture for custom planners, controllers, and perception pipelines. Find more details on [13].

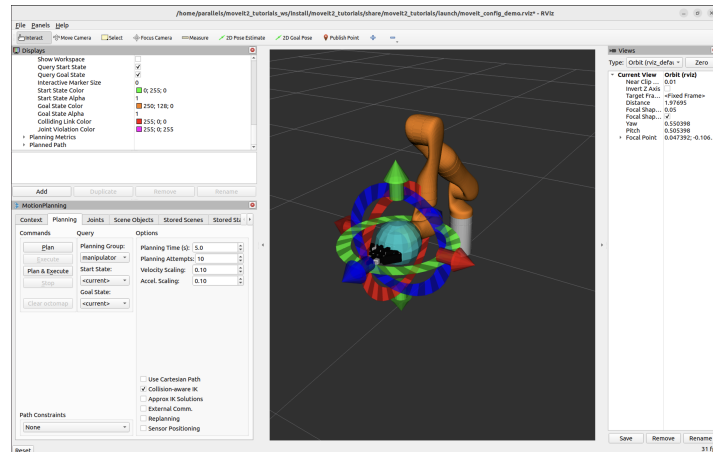


Figure 6: Moveit 2 running on RViz 2.

### 3.2.7 ArUco

ArUco markers are binary square fiducial markers used for robust pose estimation in computer vision applications (see Figure 8). The name comes from Augmented Reality University of Cordoba, representing the place where they were developed. Each marker encodes a unique ID using an internal binary grid, enabling detection and identification under various viewing conditions. Detection involves thresholding, contour extraction and perspective transformation to correct for projective distortion.

Pose estimation is performed by solving the Perspective-n-Point (PnP) problem, using the known marker size and the camera's intrinsic parameters to compute its position and orientation relative to the camera frame. ArUco is widely used in robotics for camera calibration, localization and visual servoing due to its real-time performance and robustness in cluttered environments.

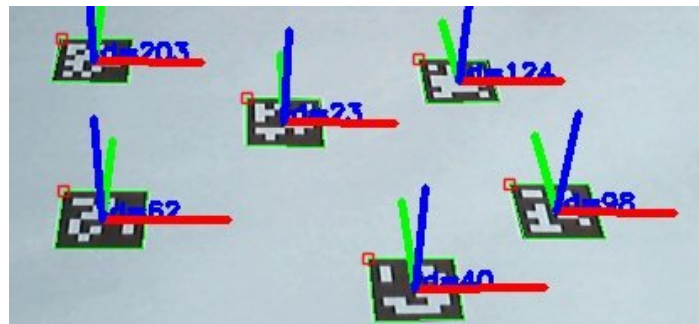


Figure 7: ArUco markers with pose estimation.

### 3.2.8 YOLOv8

YOLOv8 (You Only Look Once) is a real-time object detection and segmentation model widely used in robotics for tasks such as object recognition, tracking and scene understanding. It enables fast inference with high accuracy, making it suitable for deployment on edge devices and embedded systems.

In practical applications, YOLOv8 is often used to detect objects for pick-and-place operations, human-robot interaction, and environment mapping. The model supports instance segmentation, allowing robots to distinguish between overlapping objects. Its integration with ROS and Python APIs facilitates its use in robotic perception pipelines. Find more information in [14].

### 3.2.9 Siemens NX and PrusaSlicer

Siemens NX is 3D modeling software used to design custom 3D components. These customized 3D models, have be brought to real life using Prusa 3D printers, which use PrusaSlicer software to enable precise slicing and print preparation.

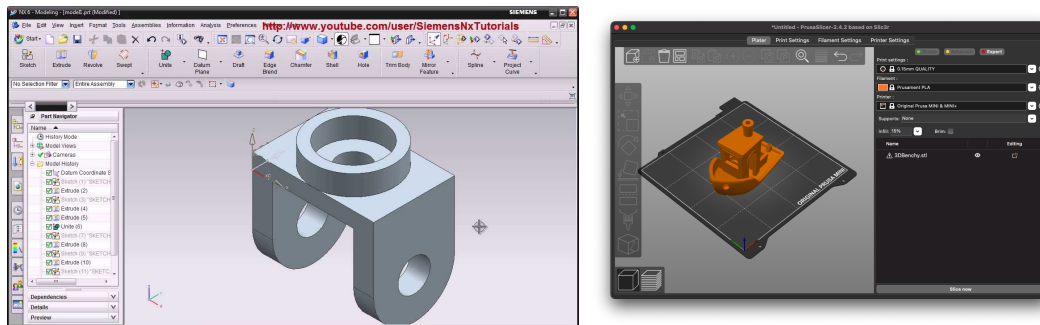


Figure 8: Siemens NX (left) and PrusaSlicer (right).

## 4 Results

This chapter explains the obtained results, which are divided in two groups: the setup (Section 4.1) and the code (Section 4.2)

### 4.1 Simplified station setup

The simplified station consists of the UR3e robotic arm with the camera placed on its end effector. As represented in Figure 9, the simplified station has the following components:

- **Landing area.** Visible area in the main platform for the camera, where the drone battery will be placed.
- **Drone battery.** This is a 3D printed simplified version of the drone. It can be translated in the two directions of the landing area and rotated in the vertical axis.
- **Fixed charging station.** This is a 3D printed structure that holds the batteries charging in a stationary place. One of the compartments from the battery charger is always empty, prepared for the battery that is coming from the drone. The rest of the compartments are filled with batteries, ready to be picked up and placed in the drone.
- **Camera.** The camera will be placed on top of the end effector. When the drone lands in the landing area, it will detect its position, so that the robotic arm can move there. It will also detect the empty compartment of the fixed battery charger and a new battery to pick up.
- **Fixed robotic arm.** A 6 DOF UR3e robotic arm will be used for doing the battery swap, using a 2FG7 gripper that allows to pick and place the batteries.
- **Batteries.** The batteries will be 3D printed components with a design that allows them to be attached in and out from the compartment.

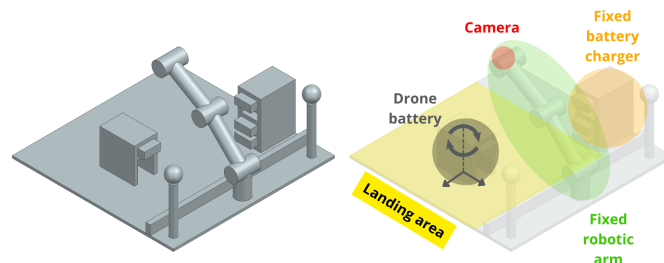


Figure 9: 3D representation of the simplified station.

In Figure 10, the complete physical setup can be observed. Its content will be explained in the next four subsections, which explain the 3D printed components (4.1.1), device distribution and communication (4.1.2) and the software setups on the PC (4.1.3) and on the teach pendant (4.1.4).

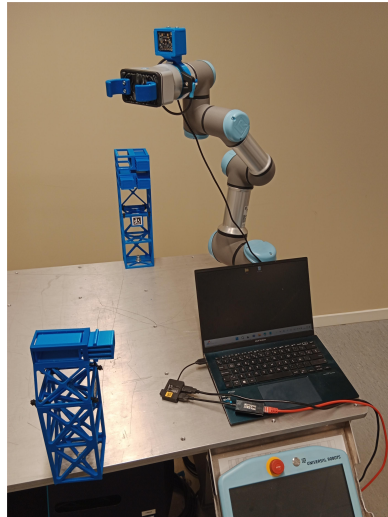


Figure 10: Simplified station physical setup.

#### 4.1.1 3D printed components

The custom 3D modeled and 3D printed components are listed below:

- **Batteries.** They have been designed so that the robotic arm can take them in and out of the compartments. See Figures 11a and 12.
- **Drone.** The drone has place for one battery, as shown in Figure 11b and a ArUco marker in its front face, for pose estimation with the camera.
- **Station.** The battery station has place for 4 batteries, as shown in Figure 11c, and a ArUco marker in the front face.
- **Camera holder.** This 3D printed part holds the camera in a fixed position in top of the UR3e's end effector, as shown in Figure 11d.
- **Gripper tips.** These customized gripper tips for the 2FG7 gripper have been created with the purpose of been able to pick up the batteries. See them in Figures 11d and 12c.

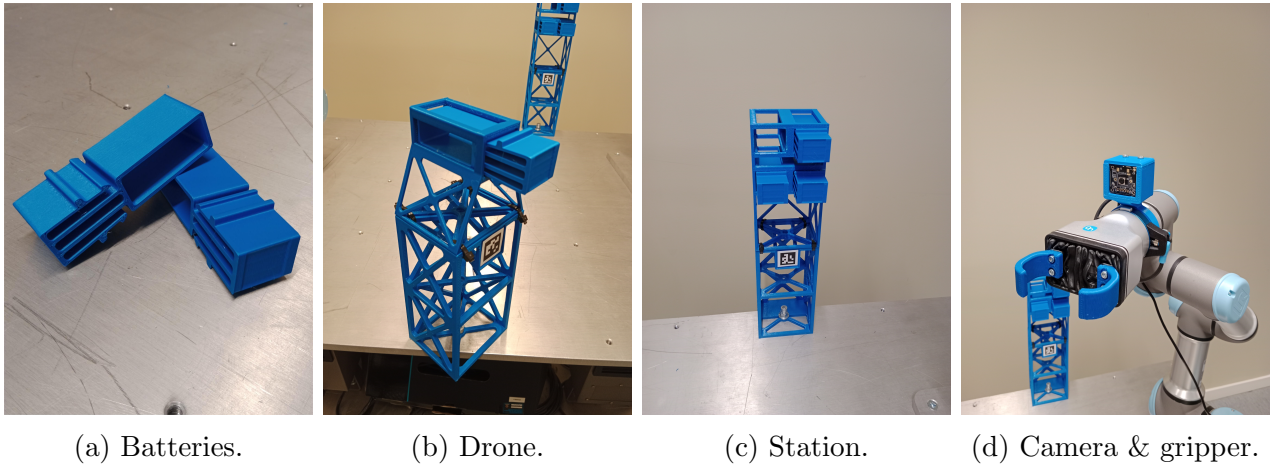


Figure 11: 3D printed components.

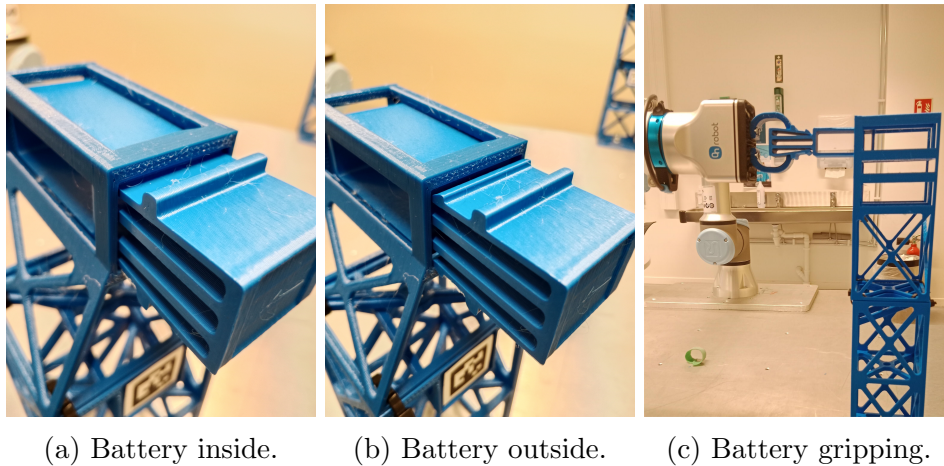


Figure 12: 3D printed batteries clip connection.

#### 4.1.2 Devices and communication

As observed in Figure 10, the devices used in the setup are the next: PC, teach pendant, UR3e with the 2FG7 gripper and the DFROBOT FIT0729 camera.

To interconnect all the devices, a small USB hub has been connected to the PC. The camera's USB 2.0 cable was plugged to the USB hub, in addition to a Ethernet to USB adapter for communication with the UR3e and gripper. Communication with the gripper is established via the same Ethernet cable that connects to the UR3e's control box, as the UR3e comes with internal connections for the gripper. See the USB hub connection in Figure 13.

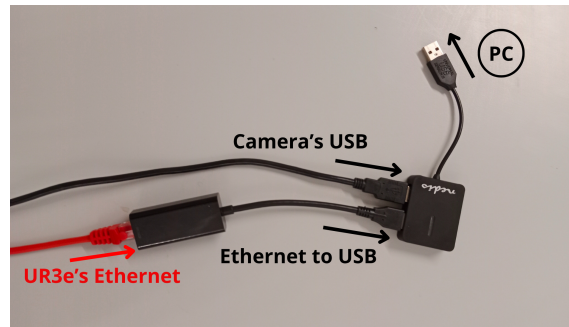


Figure 13: USB hub connection: PC, camera & UR3e (with gripper).

#### 4.1.3 Software setup on PC

Before starting the project, three main options were available to set up ROS2 Jazzy. Each of them have been tested and a final one has been chosen. Below, the three options are shown, with the comments explaining the issues experienced with each.

- **ROS2 Jazzy installation on Windows.** This option slowed down the computer a lot and often crashed.
- **ROS2 Jazzy installation on Linux (Ubuntu 24.04) using WSL.** This was tested but not selected due to IP communication issues.
- **ROS2 Jazzy installation on Linux (Ubuntu 24.04) using a VM.** First, Virtual-Box was used to run the VM, but some issues were experienced with the IP (Internet Protocol) addresses and no internet connection was available. This made the installation and access to GitHub impossible, so VMware was used instead. No further issues have been experienced with VMware.

As a result, VMware has been used to create the VM. This VM was running Ubuntu Noble 24.04 as the guest OS in a Windows host OS.

In VMware, NAT (Network Address Translation) has been used to connect to the Internet in the host OS. In addition, a bridge adapter has been set to be able to communicate with the UR3e via the Ethernet cable. Finally, a USB 3.1 adapter has also been set for the camera connection, which required connecting directly to the VM, disconnecting it from the host OS (Windows).

A problem was experienced with the USB connection for the Intel Realsense L515 camera. For some unspecified reason, the RGB camera would only be accessible via the host OS, while it was not accessible when switching it to the VM. For this reason, the L515 camera was replaced with the DFROBOT FIT0729, which following the same USB connection was available from the VM's guest OS, proving the necessary RGB data.

#### 4.1.4 Software setup on PolyScope

To connect the UR3e robot to the external control system (PC), the network connection via Ethernet has been established, as explained in the previous section 4.1.2. In addition to this, the UR3e needs to be configured from the teach pendant. This chapter will mention the needed actions for this, which are explained in the list below.

- **Network configuration.** On the teach pendant, the UR3e's network settings have to be configured so that the robot and the PC are in the same local area network (LAN). This can be done by establishing a static IP to the robot or allowing Dynamic Host Configuration Protocol (DHCP). This last action allows the robot to obtain an IP address automatically from the local network or router, and is the one that has been used.

The IP address established to the robot has been 192.168.0.100, while the computer's IP has been 192.168.0.101.

- **External control URcap installation.** For external control, the External Control URcap has to be installed on the UR3e. This is done by downloading the URcap, putting it into a pendrive and plugging it into the UR3e's control box, so then it can be installed using the teach pendant.

This URcap allows external software, such as ROS 2 nodes or other TCP/IP clients, to send joint trajectory commands to the robot via a real-time interface over a selected port, which in this case has been 50004.

- **OnRobot control URcap installation.** For using the OnRobot 2FG7 gripper, OnRobot's URcap need to be installed, which provides an interface for gripper control through URScript or dashboard commands.
- **External control activation.** Finally, after the three previous actions have been configured, the robot and the gripper can be controlled either manually through the teach pendant or through a program via script commands sent from the external controller. To allow external control, it has to be activated every single time the program is started from the external system (PC).

## 4.2 ROS2 code

This section explains the code that enables UR3e control via ROS 2 Jazzy. The code is available in Github (see reference [15]).

### 4.2.1 Overview

The ROS 2 code developed in this project provides motion control for the UR3e robotic arm with an OnRobot 2FG7 gripper and a camera mounted at the end-effector. The workspace contains two main programs to be launched:

- **Sequence Test Program.** A scripted sequence of hardcoded arm and gripper movements. It initializes the robot, executes predefined joint and linear trajectories and manipulates the gripper in open/close cycles. It includes collision avoidance for the UR3e links, the gripper, the camera, the floor and the fixed station. This program has been designed for integration and test purposes, to check that the robot moves correctly before integrating the camera.
- **Camera-Controlled Program.** Launches the robot and an attached vision system. It initializes object detection via ArUco markers and YOLOv8. The detection results are used to compute transformation matrices for positioning, followed by IK motion planning and execution, including the same collision avoidance mentioned in the previous program. As this is the main program, it will be explained more in detail, explaining its sequence of steps:
  - **Step 0.** Open the gripper.
  - **Step 1.** Move to the starting position using joint trajectory, which aligns the camera with to obtain a clear image of the landing area. Once the robot reaches the pose, call the next two camera services:
    1. The first camera service will take no input. It will take a picture and process it with the YOLOv8 model, returning the output from it.
    2. The second service will take the current robot pose as an input. It will take a picture, detect the ArUco marker's relative pose to the camera and make the necessary transformations to return the marker's pose in the world frame.
  - **Step 2.** Move to the drone to pick up the battery. This is done by using hardcoded relative positions (A, B and C) from the ArUco marker's frame, which has been obtained in the previous step. The UR3e completes the next sequence of actions to pick up the battery:
    1. Move to point A, just in front of the battery, using joint trajectory.
    2. Move to point B, which is the pick-up point, using linear trajectory from point A.
    3. Close the gripper to grab the battery.
    4. Move back to point C using linear trajectory, to take the battery out.
  - **Step 3.** Similarly to step 1, move the robot to a hardcoded pose to call the two camera services. In this case, the hardcoded pose is different from step 1, as it faces the charging station instead of the landing area.
  - **Step 4.** Similarly to step 2, the robot will complete two sequences this time: one for deployment of the battery that it already has (opening the gripper); and the other one to pick up a new battery (closing the gripper).
  - **Step 5.** Repeat step 1.
  - **Step 6.** Repeat step 2, but opening the gripper this time.
  - **Step 7.** Return to start pose, to be ready for the next drone.

### 4.2.2 Interfaces

This section explains the two nodes that have been created:

#### Node 1: Joint Trajectory Publisher.

- **Publishes to action:**

- `/scaled_joint_trajectory_controller/joint_trajectory`: This action controls the UR3e's joint positions. It takes a trajectory of joint positions as an input.

- **Subscribes to services:**

- `/compute_ik`: This service computes the IK for the given joint configuration, returning the space pose (x, y, z, roll, pitch, yaw).
- `/apply_planning_scene`: This service applies planning scenes with new collision objects, so the calculated trajectories take into account the added collision areas.
- `/plan_kinematic_path`: This service takes as input the target pose (x, y, z, roll, pitch, yaw) and returns a trajectory of joint states to achieve this point, which is linear in the joint space.
- `/compute_cartesian_path`: This service takes as input the target pose (x, y, z, roll, pitch, yaw) and returns a trajectory of joint states to achieve this point, which is linear in the spacial coordinate space.
- `/camera_process`: This service is created by Node 2 (camera server) and explained below.
- `/aruco_pose`: This service is created by Node 2 (camera server) and explained below.

- **Subscribes to action:**

- `/grip_external`: This action takes the desired gripper width as an input and communicates with it until the position is reached. It also takes maximum force and speed as input.

#### Node 2: Camera Server.

- **Serves to services:**

- `/camera_process`: Takes a picture from the camera and processes it with the YOLOv8 model. As an output of the model, it return at least two of these values (one even and one odd):
  - \* 0: Drone Empty
  - \* 1: Drone Full
  - \* 2: Station Top Left Empty
  - \* 3: Station Top Left Full

- \* 4: Station Top Right Empty
- \* 5: Station Top Right Full
- \* 6: Station Bottom Left Empty
- \* 7: Station Bottom Left Full
- \* 8: Station Bottom Right Empty
- \* 9: Station Bottom Right Full

Note that the even number represent empty compartments, while the odd one represent full compartments. See Figure 14 for a visual representation of the model's predictions.

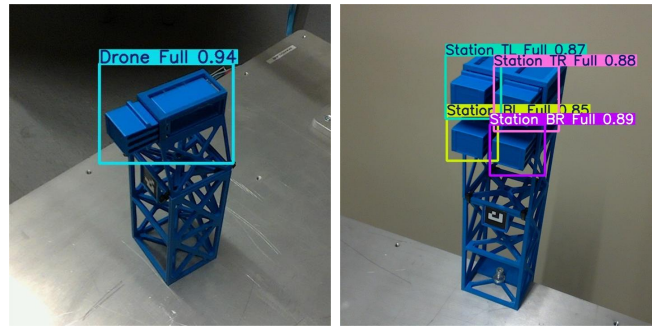


Figure 14: Visual representation of YOLOv8 model's prediction.

- `/aruco_pose`: Takes the current robot pose as an input. It will take a picture, detect the ArUco marker's relative pose to the camera and make the necessary transformations to return the marker's pose in the world frame. See Figure 15 and 16 for a visual representation of the detected pose from the marker.

The necessary transformations can be represented as follows:

$$T_{world}^{marker} = T_{world}^{tool\_tip} T_{tool\_tip}^{camera} T_{camera}^{marker} \quad (1)$$

where  $T_a^b$  represents the transformation matrix of the  $\{b\}$  frame in the  $\{a\}$  frame.

$T_{world}^{tool\_tip}$  is obtained from the UR3e, depending on the position of its joints.

$T_{tool\_tip}^{camera}$  is hardcoded.

$T_{camera}^{marker}$  is obtained from the ArUco marker by the camera.



Figure 15: Visual representation of ArUco's pose detection and established axes (red: x, green: y, blue: z).

To complete the transformation process and obtain the target pose, the **Joint Trajectory Publisher** makes the next transformation:

$$T_{world}^{target} = T_{world}^{marker} T_{marker}^{target} \quad (2)$$

$T_{world}^{target}$  is obtained from the service `/aruco_pose`.

$T_{marker}^{target}$  is hardcoded for each point A, B, C and for each value of the YOLOv8 model from 0 to 9. This means that there are 30 hardcoded  $T_{marker}^{target}$  matrices. In reality they are hardcodes as a 6 dimensional vector representing the space coordinates (x, y, z, roll, pitch, yaw). See Figure 16 for a visual representation.

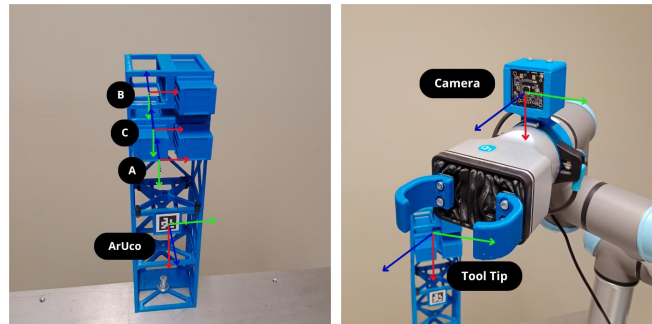


Figure 16: Visual representation of hardcoded relative poses A, B and C for YOLOv8 value 2 (red: x - green y - blue z).

### 4.2.3 Package structure and dependencies

This section lists the main packages and dependencies that have been used, separated by own developed, pulled repositories from Github or installed from binaries.

| Package                            | Functionality                                         |
|------------------------------------|-------------------------------------------------------|
| <code>ur_custom</code>             | Metapackage to install all packages with one command. |
| <code>ur_custom_bringup</code>     | Scripts to run both programs.                         |
| <code>ur_custom_description</code> | URDF, SRDF and STL files for the gripper and camera.  |
| <code>ur_custom_driver</code>      | ROS 2 Control and MoveIt planning configuration.      |

Table 1: Custom developed packages.

| Repository                        | Purpose                                                       |
|-----------------------------------|---------------------------------------------------------------|
| <code>onrobot_gripper_ros2</code> | Interface for controlling the OnRobot 2FG7 gripper via URCap. |

Table 2: External packages pulled from GitHub.

| Package                                                             | Functionality                                             |
|---------------------------------------------------------------------|-----------------------------------------------------------|
| <code>ros_jazzy_ur_robot_driver</code>                              | UR ROS 2 driver for hardware interface and communication. |
| <code>ros_jazzy_ur_description</code>                               | Official UR3e robot description.                          |
| <code>ros_jazzy_moveit</code>                                       | Motion planning with OMPL, collision checking, IK.        |
| <code>ros_jazzy_rviz2</code>                                        | 3D visualization of the robot and environment.            |
| <code>ros_jazzy_ros2_control</code>                                 | Hardware interface abstraction and control manager.       |
| <code>ros_jazzy_xacro</code>                                        | Robot description macros for URDF generation.             |
| <code>ros_jazzy_joint_</code><br><code>trajectory_controller</code> | Joint-level motion command execution.                     |

Table 3: Packages installed from binaries.

#### 4.2.4 Observations

This chapter will summarize the observations for each of the two programs:

- **Sequence Test Program.**
  - Completed the sequence successfully.
  - Collision avoidance accomplished.
  - Sometimes it takes some time to find a path or even fails.
- **Camera-Controlled Program.**
  - Failed to start the program and complete the first step.

## 5 Discussion

The development of the drone battery swapping system using the UR3e robotic arm and ROS2 Jazzy achieved partial success.

As mentioned in the end of Section 4, the Sequence Test Program successfully executed hard-coded joint and linear trajectories with collision avoidance, demonstrating the UR3e's capability to navigate the simplified station's workspace.

The 3D-printed components, including batteries, gripper tips, drone and charging station, functioned as intended, enabling reliable pick-and-place operations with the OnRobot 2FG7 gripper. The integration of the DFROBOT FIT0729 camera via a USB hub and VMware-based Ubuntu 24.04 environment proved effective for hardware communication, overcoming initial issues with the Intel Realsense L515 camera's USB connectivity in the virtual machine.

The occasional path planning failures suggest that the configuration space sampling in RRT algorithms was not always robust, possibly due to insufficient tuning of step size  $\alpha$  or collision cost parameters.

On the other hand, the Camera-Controlled Program failed to complete its first step, highlighting significant challenges in the vision-based control system. This does not mean that the camera detection was not working, as the failure was given at the first step of the program, which was to open the gripper and move to the start pose. Probably the issue is related with communication among the nodes or with the structure of the code for this program.

In top of that, the use on YOLOv8 and ArUco markers for pose estimation introduced complexities in real-time processing and transformation calculations. The hardcoded relative poses (A, B, C) for battery manipulation, while theoretically sound good, are sensitive to environmental variations, such as lighting and marker occlusion, which could cause further problems.

Additionally, the hardcoded relative positions and frames, have not been taken accurately enough, which will cause precision issues. At the same time it would be a good idea to review that the transformation matrices were deploying the correct joint configurations.

## 6 Conclusion

As a conclusion, this project successfully developed a simplified prototype of a drone battery swapping station using a UR3e robotic arm controlled via ROS2 Jazzy. The physical station, comprising 3D-printed batteries, a drone, a charging station, and a camera holder, was effectively set up and integrated with the UR3e, OnRobot 2FG7 gripper and DFROBOT FIT0729 camera.

The Sequence Test Program demonstrated reliable motion planning and collision avoidance, validating the hardware and basic ROS2 control architecture. The use of ROS2 nodes, including the Joint Trajectory Publisher and Camera Server, established a foundation for modular control, leveraging services and actions for trajectory execution and vision processing.

Despite these achievements, the Camera-Controlled Program's failure to initialize highlights limitations in the vision-based control system, likely due to challenges in the code.

The project met its objective of setting up a functional hardware platform but only partially achieved autonomous control, as vision integration remains incomplete. The theoretical background provided a solid framework for kinematics and motion planning, but practical challenges in software robustness and environmental adaptability require further refinement. Overall, the project lays a strong foundation for an autonomous battery swapping system, with clear pathways for improvement.

## 7 Future Work

Future work should address the limitations observed in the Camera-Controlled Program to achieve fully autonomous battery swapping.

First, enhancing collision avoidance to include the drone's dynamic position is critical and it wasn't included in the completed work. This involves modeling the drone as a moving obstacle in the configuration space  $\mathcal{C}_{\text{obs}}$ , using real-time updates from the camera to adjust RRT or PRM paths dynamically.

Second, improving precision in battery manipulation is necessary, particularly for the 2FG7 gripper's pick-and-place operations. This could involve calibrating the hardcoded relative poses (A, B, C) with more accurate measurements and making sure that the pick and place of the battery is completed successfully. In addition, adaptive algorithms that account for environmental variations could be used, such as lighting or marker misalignment, potentially using machine learning to refine YOLOv8 predictions.

Third, robust integration of the vision system is essential. Optimizing the Camera Server node to handle YOLOv8 and ArUco processing in parallel, possibly through asynchronous service calls or GPU acceleration, could mitigate synchronization issues. Additionally, replacing hardcoded transformation matrices with adaptive pose estimation techniques, such as iterative closest point (ICP) for markerless detection, could enhance reliability.

Finally, improving the ROS2 architecture's error handling, through detailed logging and fault-tolerant node communication, will support debugging and scalability.

These improvements will enable a fully autonomous, precise and reliable drone battery swapping system, building on the current prototype's foundation.

## References

- [1] Kevin M. Lynch and Frank C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge, United Kingdom: Cambridge University Press, 2017.
- [2] *Universal Robots UR3e Collaborative Robot Arm*. 2025. URL: <https://www.universal-robots.com/products/ur3e/>.
- [3] *2FG7 Parallel Gripper*. 2025. URL: <https://onrobot.com/en/products/2fg7>.
- [4] *LiDAR Camera L515*. 2025. URL: <https://www.intelrealsense.com/lidar-camera-l515/>.
- [5] *FIT0729 Camera Module*. 2025. URL: <https://no.rs-online.com/web/p/raspberry-pi-cameras/2473234>.
- [6] *Robot Operating System*. 2025. URL: <https://www.ros.org/>.
- [7] *Beginner CLI Tools Documentation*. 2025. URL: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools.html>.
- [8] *Windows Subsystem for Linux*. 2025. URL: <https://learn.microsoft.com/en-us/windows/wsl/>.
- [9] *VirtualBox*. 2025. URL: <https://www.virtualbox.org/>.
- [10] *VMware Workstation*. 2025. URL: <https://www.vmware.com/products/workstation-pro.html>.
- [11] *Ubuntu 24.04 LTS*. 2025. URL: <https://ubuntu.com/>.
- [12] *RViz Tutorial*. 2025. URL: <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-Main.html>.
- [13] *MoveIt 2 Tutorials*. 2025. URL: <https://moveit.picknik.ai/main/doc/tutorials/tutorials.html>.
- [14] *YOLOv8 - Ultralytics*. 2023. URL: <https://yolov8.com/>.
- [15] *UR3e-Control: ROS 2 Jazzy workspace for UR3e robotic arm*. 2025. URL: <https://github.com/jonurce/UR3e-Control>.